一套应用怎样连接起来

“打印机离线”进入程序后，先是一段文字，后来成为一次 HTTP 请求、一张数据库工单和一项合作方派单。处理人员还可能为它请求一段政策说明。它们都与同一件事有关，却不是同一个对象：回复丢失了，工单仍可能存在；模型写了“可以关闭”，数据库也不该因此改变。

这本书把前六册的工具放回一套完整的合成支持应用。我们先追一次提交，再调查失败、组织证据，最后看运行实例或程序版本变化后怎样继续服务。需要建立的是可以沿着具体对象找回的关系，完整函数名、字段拼写和命令参数可以就近查用。

第一次只看这里

直接进入 X01。第一次只追一张工单：输入经过哪些责任边界，最终结果怎样留下可恢复证据。

本册的详细关系图、章节路线和查用说明放在全书后部。遇到‘这一步和前后哪里相连’时再回看，不需要在第一轮先读完。

目录

第一次只看这里 1

X01 | 一张工单怎样从文字变成可以读回的事实？ 3

X02 | 同样返回 503，为什么处理方式不同？ 10

X03 | 怎样证明重放没有多建一张工单？ 14

X04 | 换掉运行实例以后，怎样确认还能服务和恢复？ 19

关系收束 | 从一个现象找到下一项有用证据 26

本册总览（需要时再回看） 28

本册知识结构总览 29

X01 | 一张工单怎样从文字变成可以读回的事实?

本章先判断：沿着一张工单向前追时，每次交接交出了什么、谁接管状态，以及哪份证据足以让结果继续向下游走。 先追一条成功路线，再在同一条链上定位最早断点。

先追客户端值怎样经过 API 合同和事务变成可读事实，再看外部安排如何改变后续状态。

用户提交 Printer-7 的故障，期望至少得到一个明确结果：“工单已经保存，编号是多少，目前是否取得合作方安排？”这是本章的成功条件。政策解释和备份恢复随后才加入，先把创建走到结果。

同一个 3，怎样离开客户端进程?

客户端取得优先级时，值是字符串 `"3"`。它需要先把文字变成整数，再检查是否处于允许的 1–3 范围。实际 `client.py` 中的核心逻辑为：

PYTHON
```
def parse_priority(priority_text):
    try:
        priority = int(priority_text.strip())
    except (AttributeError, ValueError):
        raise PriorityInputError(
            "priority must be the text form of 1, 2, or 3"
        ) from None
    if priority not in {1, 2, 3}:
        raise PriorityInputError("priority must be 1, 2, or 3")
    return priority
```

这是源文件的函数切片；`PriorityInputError` 在同一文件中定义为 `ValueError` 的子类。`strip()` 允许输入首尾有空白，`int()` 完成转换，集合成员判断负责业务范围。`"urgent"` 在转换时失败；`"-3"` 能转成 -3，却会在范围检查时失败。两者都不会走到发送 HTTP 的下一句。

对于 `"3"`，函数交回整数 3。`create_ticket()` 再把它放入四个字段组成的请求体：

JSON
```
{
  "device_id": "Printer-7",
  "subject": "打印机离线",
  "priority": 3,
  "data_classification": "synthetic"
}
```

JSON 中的 `3` 没有引号；`"3"` 则会表示字符串。这个差别进入了 API 的合同，不只是代码看起来不同。

发送时，HTTPX 的 `json=` 参数接收客户端的 Python 字典，编码出请求体。另一端读取字节并解析出自己的对象。两个进程不会共享客户端内存里的同一个 dict。

本次交换可写成下面的 HTTP 形状。端口使用示例值；真正运行时以该实验的实际监听端口为准。

```HTTP
POST /tickets HTTP/1.1
Host: 127.0.0.1:18080
Content-Type: application/json
X-Request-ID: R-X-42
Idempotency-Key: ticket-create:Printer-7:offline-001
```

body 就是上面的四字段 JSON。`POST /tickets` 告诉 API 要创建工单；`X-Request-ID` 帮助关联本次处理；`Idempotency-Key` 标识这次稳定的创建意图。结果未知而重发时，创建 key 和业务内容保持不变，请求编号换新。这样才可能同时说明“又传了一次”和“仍是那一件事”。

客户端已经检查过，API 为什么还要检查？

任何调用者都可能直接向 API 发请求，不一定经过这个客户端。服务端必须根据自己的合同重新判断输入，不能因为某个页面曾经限制输入就信任所有请求。

`TicketCreate` 是服务端希望取得的已验证对象。其严格规则包括：

输入内容	API 实际要求
device_id	3–40 个字符，限定字母、数字和连字符
subject	3–120 个字符
priority	整数 1–3；不把字符串自动当整数接受
data_classification	固定文字 synthetic
额外字段	拒绝

因此，直接发送 `"priority":"3"` 会得到 422；正常客户端发送整数 3 才符合这条类型约定。FastAPI 完成输入解析与校验后，才会把 `TicketCreate` 交给创建 endpoint，即负责这个路径的业务函数。

此时只是 API 已经有了一个可用的内存对象，数据库还没有新的工单。`validation_passed` 不能翻译成“工单已保存”。接下来的事务才决定业务事实是否出现。

第一笔事务为什么同时保存工单、意图和事件？

API 先核对必要的数据读取与 schema 版本条件，然后调用 `store.create_intent()`。这里的 intent 表示一次准备执行的业务意图。程序在 `tickets` 行上保存 `create_key` 与规范化输入的摘要；它们使后续请求能够识别同一意图，而不是每收到一次 POST 就创建一张新票。

空数据库把下一个编号记为 42。当前事务中的关键变化是：

```
BEGIN IMMEDIATE
→ 根据 create_key 查询：尚无旧意图
→ 读取 next_ticket_number=42
→ INSERT tickets：T-42，status=open，assignment_status=pending
→ INSERT ticket_events：ticket_created，request_id=R-X-42
→ 把 next_ticket_number 更新为 43
→ COMMIT
```

`BEGIN IMMEDIATE` 在开始时申请 SQLite 的写事务。工单、创建意图信息、创建事件和编号变化都在同一次事务中。这里没有单独的“幂等记录表”：key 与摘要就是 tickets 的两列。

为什么事件也放在里面？假如工单保存了，记录其来源的事件却没保存，后来的人会看到一张缺少处理依据的票。若事件写入被受控约束拒绝，整个事务退出时撤回，tickets 无新行、events 无新行、下一个编号仍是 42。测试可以在事件 INSERT 处制造失败来检验这件事。

事务提交以后，T-42 才成为别的请求可以读到、进程重启后应仍能找回的事实。`ticket_transaction_committed` 日志也只能在确认提交成功之后产生；把这条日志提前打印并不能让数据库提交。

> **第一次学到这里先停。** 这条综合主线先到“输入经过客户端校验、API 合同和本地事务，变成可读回的 T-42”。合作方派单、AI 草稿、恢复与回滚是在这条已保存事实之上继续增加责任边界；第一次阅读不必同时追完所有下游。

open 与 pending 为什么会同时成立？

它们描述两个对象的问题。工单 `status=open` 表示这件故障还在处理；`assignment_status=pending` 表示合作方派单结果还没在本地确认。工单可以存在，同时没有安排结果。

源码中的 SQLite 列名是 `status`。API 对外把它映射为 `ticket.ticket_status`，使读者不必猜一个孤立的 status 指哪件事。映射改变字段名称，没有改变数据所有者。

这时应用向模拟合作方发出第二份 HTTP 请求，其 body 只有：

```
JSON
{"ticket_id":"T-42","device_id":"Printer-7"}
```

header 另带 `Idempotency-Key: assignment:T-42` 和用于关联本次处理的 `X-Request-ID: R-X-42`。不发送 subject、priority，不等于没有发送关联 header。合作方创建 key 与本地工单创建 key 分属两种意图，不能互换。

这个网络调用发生在第一笔事务结束后。应用没有持有 SQLite 写事务等待合作方网络回复。相应地，第一笔事务不能在稍后 timeout 时跨网络撤销合作方已经做完的事情。

合作方确认以后，为什么还要第二笔事务？

合作方拥有自己的结果，当前返回例如：

```
JSON
{"accepted":true,"assignment_id":"PA-1"}
```

adapter 先检查 HTTP 状态，再解析 JSON、校验 accepted 和 assignment_id 的类型与形状。拿到这一结果，只能说明调用方获得了一项可用的合作方确认；本地 T-42 行还没因此自动更新。

应用调用 `finalize_assignment()`，在第二笔事务中把 assignment_status 从 pending 改为 partner_confirmed，保存 PA-1，并加入 assignment_confirmed 事件。两项写入一起提交。工单本身的 status 仍是 open；有人接单不表示打印机已经修好。

正常创建的公开结果是 HTTP 201。下例省略时间、版本和设备等已在工单对象中的字段，展示当前需要读懂的部分：

```
JSON
{
  "request_id":"R-X-42",
  "local_saved":true,
  "partner_confirmed":true,
  "idempotency_replayed":false,
  "ticket":{
    "ticket_id":"T-42",
    "ticket_status":"open",
    "assignment_status":"partner_confirmed",
    "partner_assignment_id":"PA-1"
  }
}
```

201 属于本次 HTTP response；open 属于工单；partner_confirmed 属于派单子流程。顶层 partner_confirmed 从已保存的派单状态推导，不能只看刚收到的合作方回复就填写 True。

随后调用 `GET /tickets/T-42`，API 查询本地行并返回。GET 不重新派单，也不调用模型。读回当前公开工单字段，才把“API 告诉我保存了”与“下一次读取确实能找到它”接起来。创建 response 不是存储位置，它只是 API 对已确认事实的公开表达。

另一个请求为什么只产生草稿?

工单已经保存，处理人员现在想知道批准恢复前需要哪些条件。这是另一项任务，用新的 `R-X-43` 调用 `POST /tickets/T-42/policy-draft`。

应用读出 T-42，并取得固定资料 `policy-2026 §4`：

> 处理人员必须先确认设备处于离线状态；
> 批准任何恢复操作前，必须备份当前配置。

这是完整的合成政策，不是某个现实机构的服务承诺。它与工单主题也不是同一种事实：主题说明“用户报告了打印机离线”，政策说明“操作前应满足什么条件”，都没有证明设备此刻已经被检测或完成备份。

Fake Model 接收本次允许的工单字段与这段资料，提出草稿。应用依次检查结构、引用是否属于本次资料集合、固定文字是否符合本实验的窄政策条件，再处理工具建议。接受结果写为 `policy_draft.status=accepted_draft`，表示可以交人核对的草稿，没有给工单增加“已完成”事实。

检索不到适用资料时，返回 abstained，模型调用为零。引用了未检索到的来源时，返回 needs_human。若模型提出 close_ticket，本应用仍只有 ticket:read 权限，结果为 needs_human / tool_permission_denied，工具调用与效果都为零，ticket_status 保持 open。

这里的检索是对封闭资料的关键词和适用条件筛选，claim 检查也是固定文字条件。它们让“先找依据、再检查候选、单独授权动作”的位置可见，不证明通用语义理解或真实模型质量。需要扩大到真实资料时，要回到 E 册的资料准备、用途验证与固定评测。

先读关系，想运行时再准备实验副本

没有运行环境时，可以先沿正文中的输入、实现和预期状态追踪；运行再增加对应环境的实证。需要动手时，从资料文件夹中可见的 `02_配套代码_F与综合应用 /examples/x_cross_layer/support-ticket-system` 整个复制到自己的独立实验目录，随后在含有 `requirements.txt`、`app.py` 和 `walkthrough.py` 的目录中操作。`02_配套代码_F与综合应用` 是底层 `02_配套练习_FX新版` 的入口，二者指向同一份配套；后文文件内若出现底层名称，不需要再复制另一份。不要把新旧两份同名配套混合，也不要直接在参考资料目录制造运行状态。

主机实验要求 CPython 3.13。先在实验副本中检查下面第一行，确认版本；`py` 是解释器启动器，`-3.13` 指定版本，不是本项目的模块。已有 `.venv` 时不要覆盖，应直接检查其解释器；若不符合本例条件，另建干净实验副本。

```powershell
py -3.13 -c "import sys; print(sys.executable); print(sys.version)"
```

没有 `py` 启动器，不等于没有 Python。可以改用 A09 中已核对的 **CPython 3.13 解释器完整路径**，形状为 `& '<实际解释器路径>' -c "import sys; print(sys.executable); print(sys.version)"`；引号内必须替换为自己电脑上真实存在的路径，不要输入尖括号。确认后，把下面的 `py -3.13` 同样替换为 `& '<实际解释器路径>'`。若只有其他版本或尚未找到可用解释器，先沿正文读关系，不继续环境创建，也不把其他版本的运行称为本例声明环境的验收。

在不存在 `.venv` 的新实验副本中建立环境，再把它的解释器路径保存在本次 PowerShell 会话的 `$xPython` 中：

```powershell
if (Test-Path -LiteralPath .\.venv) { throw '已有环境，请先核对它，不覆盖。' }
py -3.13 -m venv .venv
if ($LASTEXITCODE -ne 0) { throw '虚拟环境没有建立。' }
$xPython = Join-Path (Get-Location) '.venv\Scripts\python.exe'
& $xPython -m pip install -r .\requirements.txt
if ($LASTEXITCODE -ne 0) { throw '依赖安装没有完成。' }
& $xPython -m pip check
if ($LASTEXITCODE -ne 0) { throw '依赖关系不一致。' }
```

依赖安装需要取得软件包；之后的章内 trace 使用临时 SQLite、进程内 HTTP 测试和 Fake，不需要真实合作方或模型账号。实验数据库由程序放在系统临时目录并在结束后关闭、清理，环境和源码则留在刚复制的实验副本中。后面的 `$xPython` 命令沿用本次会话；重新打开 PowerShell 时，先回到同一实验副本目录，再执行上面给 `$xPython` 赋值的那一行，无需重复创建环境。

看见同一程序里的这些交接

实验副本提供 `walkthrough.py`，每次运行建立一份空临时数据库，使用本地模拟合作方和 Fake Model：

```powershell
& $xPython .\walkthrough.py success
if ($LASTEXITCODE -ne 0) { throw '成功路线与预期不一致。' }
```

关键结果应为：创建 201、显式重放 200；一张 T-42，优先级 3，工单 open，派单 partner_confirmed；provider_calls=1，provider_effects=1，model_calls=1，tool_calls=0。重放部分在 X03 继续解释。

这个 trace 使用 TestClient 在进程内进入真实 API，SQLite 真正提交，外部 HTTPX 传输由 MockTransport 接住。它没有经过真实监听端口。这样先观察程序如何协作，随后再用双进程主机实验补端口与进程的证据。

到这里可以拿起任一中间对象说明它的下一位接收者：整数交给客户端组装请求；JSON 交给 API；验证对象交给业务函数；已提交工单交给查询和派单；合作方确认交给第二笔事务；公开结果交给调用者。政策草稿则沿另一条请求交给人，始终没有冒充工单事实。

X02 | 同样返回 503，为什么处理方式不同？

本章先判断：同一个状态码出现时，失败发生前已经留下了哪些本地和合作方事实；下一步应查证、停止、重试还是转人工。

现在沿刚才的创建路线制造失败。调用者看到 503，想知道能否再次提交。直接回答“503 就重试”会漏掉最重要的一件事：失败发生之前，本地和合作方分别已经保存了什么？

本章比较两次互相独立的实验。它们都从空数据库开始，固定输入仍是 Printer-7、打印机离线、整数 3。两次的 T-42 只是各自数据库的第一张票，不能把一条实验留下的行带进另一条。

第一种 503：连本地创建都没开始

让另一个 SQLite connection 持有排他锁，再向 API 提交合法请求。API 已经通过输入验证，随后检查必要的数据读取条件；读取受锁阻塞并达到本地等待限制，于是停止创建。

这个受控场景在 `walkthrough.py write-blocked` 中运行：

```powershell
& $xPython .\walkthrough.py write-blocked
if ($LASTEXITCODE -ne 0) { throw '写入前失败路线与预期不一致。' }
```

可观察到的先后关系是：

```
R-X-52 到达 API
→ validation_passed
→ 必要数据库读取失败
→ ticket_write_precondition_failed：database_unavailable
→ response_completed：503

释放实验锁后读取：tickets=0，next_ticket_number=42
provider_calls=0，provider_effects=0
```

最后一项已确认的前进是输入合法；下一项本应是数据条件可用，实际在这里停止。没有工单是预期结果，不是另一个新故障。

为什么事件名不是 ticket_transaction_failed？因为这次锁首先挡住了创建之前的读取检查，还没有进入 create_intent 的事务。若另一个实验让读取检查通过，再使事务内写入失败，才会走到事务失败事件。同样的公开原因可以来自相邻位置，日志要记录实际发生的那一段。

“没有 commit 日志”本身不能独立证明 rollback：日志也可能漏记、收集失败或查询条件错了。这里还在释放锁后核对了正确数据库中的行数和计数器，再结合 provider 调用为零，才得到闭合证据。

故障解除后，本项目允许用相同 key 和完全相同 payload 再次尝试。这时没有旧意图，下一次仍可创建本实验的 T-42。这个规则来自当前程序明确处理的持久化失败，不推广为任意代理或第三方返回 503 都允许安全重试。

第二种 503：工单与合作方结果都已存在

这次让第一笔事务和合作方调用正常完成，只在第二笔事务保存之前制造失败。应用刚取得 PA-1，却没能把派单确认写入本地。响应的 detail 是：

```JSON
{"code":"assignment_result_not_persisted","ticket_id":"T-42"}
```

现在三份事实为：本地有 T-42/open/pending；合作方有一项 PA-1；调用者收到了 503。它们并不矛盾，分别属于不同位置。

> 核心事务已 COMMIT
> → 模拟合作方已产生 PA-1
> → 第二笔事务未保存确认
> → 本地 assignment_status 仍 pending
> → 503 明确指出结果尚未持久化

不能把“API 失败”解释成“工单不存在”，也不能把 PA-1 的成功回复解释成“本地已经 partner_confirmed”。这正是必须保留 pending 的原因：应用承认有一个尚未收束的交接。

`walkthrough.py pending-recovery` 会在同一次独立实验中继续恢复：

```POWERSHELL
& $xPython .\walkthrough.py pending-recovery
if ($LASTEXITCODE -ne 0) { throw 'pending 恢复路线与预期不一致。' }
```

程序解除注入的保存故障，换新的 request ID，再传相同创建 key 和 payload。create_intent 识别旧行，返回同一个 T-42，且 assignment_status 仍 pending。应用据此继续派单交接；合作方收到相同 assignment:T-42 和相同内容，交回已经保存的 PA-1，不创建第二项效果。

第二笔事务这次提交成功。HTTP 返回 200，idempotency_replayed=true，表示本请求在继续原有创建意图。最终应有一张工单、两次合作方调用、一项合作方效果，assignment_status=partner_confirmed。**两次调用不等于两次安排。**

原有 D 册为其独立案件应用保存了版本化 canonical payload，用于判断跨请求恢复资格。X 这里依据自身不可变的工单字段、create_key 与输入摘要恢复这条受限派单路线；不能把 X 的字段集合说成 D 的全部 payload 合同已经照搬。

外部失败为什么有时返回 202?

把合作方改成每次都返回 500，创建仍先保存 T-42。adapter 在有限次数和预算内尝试，无法取得可用确认后，第二笔事务写入 assignment_status=manual_review，同时加入人工接管行和对应事件。

> 本地 T-42 已保存
> → partner 两次 500，无确认结果
> → manual_review + manual_handoffs 一起提交
> → HTTP 202，local_saved=true，partner_confirmed=false

用户的最低目标“保存工单并让后续人员能够接手”成立；自动派单目标没有完成。所以它是一个明确降级结果，而不是把失败偷偷换成成功。

`walkthrough.py degraded` 应得到创建 202、重放 200、两次 provider calls、零次 provider effects，以及一条人工接管记录。重放已经收束的 manual_review 意图只读回旧结果，不重新消耗一轮重试预算。

本例的自动尝试最多两次。每次开始前重新计算剩余时间，传入不超过剩余时间的 timeout，退避等待也受预算限制。HTTPX 的 timeout 约束连接、读、写和连接池等传输阶段；它不是对整个业务调用强制执行的统一墙钟闹钟。代码还能在返回后拒绝超出预算的结果、停止后续尝试，不能声称正在进行的所有底层阶段累计绝不会超时。

timeout 又与这里的 500 不同。500 已经是收到的回复；timeout 表示当前阶段没有及时得到所需结果，合作方可能已经做了。本地人工接管记录不能据此写“对方肯定没创建”。需要核对的是那一次派单意图，不是盲目再建一个新 key。

先判断有没有请求，再判断请求停在哪里

503 的两条路线都进入了业务函数。若输入更早就被拒绝，调查应停在更早的位置。

实际观察	当前能定位到哪里	下一份有用证据
parse_priority 对 urgent 报错，HTTP 调用为零	客户端转换	当前文字与转换入口
HTTPX 报 ConnectError，没有 response	目标连接边界	实际 URL、端口、监听进程与服务端接收情况
priority 为字符串，API 返回 422	输入合同	本次 request body 与字段错误位置
validation 通过，数据读取失败，503	创建之前的数据条件	正确数据库路径、锁与该次失败事件

实际观察	当前能定位到哪里	下一份有用证据
本地 pending，合作方有结果，503	第二笔事务交接	本地行、稳定意图及合作方已有 receipt
本地 manual_review，202	可选派单降级	人工接管原因与后续处理者

这张表帮助选择下一项调查，不要求每次逐行运行全部工具。若已看见 422 且没有业务入口事件，就没有依据先改 SQL 索引。若没有取得 HTTP response，也不能把 transport exception 编造为一个服务器 500。

定位应从已知输入出发比较期望与实际，但实际收集证据可以从最容易观察的位置开始。客户先给了一条 response，就先确认它属于哪次请求，再向前寻找最后成功与首次偏离。调查顺序不必等于执行顺序。

另一条分支失败，会不会改变已经保存的工单？

假设 R-X-43 的政策请求返回 HTTP 200，但 policy_draft.status=needs_human，reason_code=citation_not_retrieved。这表示 API 成功交回了一个“候选不能采用”的业务结果，不表示模型内容可用。

本次检索得到 policy-2026 §4，模型却引用其他来源。最早失去依据的位置在候选引用验证，工单 T-42 不应因此被删除或改状态。工具越权也是如此：`walkthrough.py ai-permission` 会产生 needs_human / tool_permission_denied，tool_calls=0。

另一个容易混淆的位置是健康检查。应用进程可以回答 /live，但 expected schema=2、数据库 metadata 仍记 1 时，/ready 返回 503，写入也被拦住。这个 X 应用的原因码叫 schema_version_mismatch；F 中较小项目可能使用不同字符串。共通的是期望与实际不相容，不是各示例自动共用一个 reason-code 命名空间。

看到失败后，先保留本次 request ID、输入形状、公开原因和对应持久事实。重启、修改 retry 或换配置都是会改变现场的动作；它们应当针对已经定位的关系。修复完成后，再重跑当初暴露问题的最小场景，检查那一处偏离是否消失。

X03 | 怎样证明重放没有多建一张工单？

调用者没看见结果，重新发了一次请求。API 返回 200 和 T-42。只凭这两个值能否确认“没有重复创建、没有重复派单”？还不够：程序可能错误地新建了隐藏记录，再返回旧编号；也可能本地只有一张票，却向外部派了两次单。

现在只保护一个具体承诺：**同一创建意图使用同一 key 和完全相同的输入重放，应该得到原工单，且不增加已经完成的业务效果。** 需要的证据由这项承诺决定，不按工具数量决定。

两次请求怎样指向同一件事？

普通成功实验从空数据库开始：

> 第一次：R-X-42
> key=ticket-create:Printer-7:offline-001，payload=P
> → 201，T-42，idempotency_replayed=false
>
> 第二次：R-X-REPLAY-42
> key仍相同，payload仍为P
> → 200，同一T-42，idempotency_replayed=true

两次 request ID 用来区分两次 HTTP 处理，key 连接稳定意图。它们不是用户身份，也不是访问权限。为所有调用统一填一个 request ID，会让日志难以区分第一次创建和后来的重放；每次换一个 key，则会把同一业务意图变成多个新意图。

API 在第一笔事务中按 create_key 查询旧行，比较本次规范化 payload 的摘要。摘要相同并且旧 assignment 已经收束时，直接返回已保存工单，不进入 partner.assign。若摘要不同，返回 409，原行保持不变。

改变主题后仍用旧 key，不是“同一请求再试一次”。例如把“打印机离线”改成“打印机仍然离线”，内容已不同，必须先判断这是修改现有工单还是新的业务动作；不能让幂等实现默默覆盖原意图。

四份证据怎样互相补足？

对于已经 partner_confirmed 的普通路线，最小充分核对可以是：

观察位置	本例预期	排除什么误解
创建与重放 response	201→200，同一 T-42，replayed 标记改变	重放不能冒充新的创建
tickets 与创建事件	只有一张票、一条 ticket_created	同一结果文字不能遮住重复持久化

观察位置	本例预期	排除什么误解
provider calls/effects	都保持 1	本地一张票不代表外部只有一项安排
第二次请求日志	ticket_intent_replayed，无新 commit/partner 事件	说明这次经过的控制路径

它们彼此支持，却没有互相替代。response 说明公开合同；查询说明当前数据库；Fake 计数说明当前受控外部效果；日志说明应用怎样记录这次运行。

pending 恢复则使用另一组预期：provider_calls 可以从 1 增到 2，provider_effects 仍是 1，因为第二次请求确实继续了尚未收束的外部交接。若把“所有重放都不能增加调用数”作为全局规则，反而会错误拒绝 X02 的安全恢复路线。

用一段可运行实验把这些断言接在一起

下面程序保存为实验副本中的 `x03_replay_check.py`。它使用配套 `walkthrough.synthetic_system()` 准备已经说明的应用、临时数据库、模拟合作方、模型、工具和日志流；退出 with 时关闭连接并清理临时数据库。辅助函数只负责建立这些对象，创建、重放和检查都在下面可见。

PYTHON

```
import json

from walkthrough import BODY, headers, synthetic_system

with synthetic_system() as (client, store, partner, model, tool, logs):
    first = client.post(
        "/tickets", json=BODY, headers=headers("R-X-42")
    )
    replay = client.post(
        "/tickets", json=BODY, headers=headers("R-X-REPLAY-42")
    )
    assert first.status_code == 201
    assert replay.status_code == 200
    assert replay.json()["idempotency_replayed"] is True
    assert replay.json()["ticket"] == first.json()["ticket"]
    assert store.count_tickets() == 1
    assert partner.call_count == 1
    assert partner.effect_count == 1

    events = store.events("T-42")
    assert sum(e["event_type"] == "ticket_created" for e in events) == 1
```

PYTHON · 续

```
    fetched = client.get(
        "/tickets/T-42", headers={"X-Request-ID": "R-X-44"}
    )
    assert fetched.json()["ticket"] == first.json()["ticket"]

    records = [json.loads(line) for line in logs.getvalue().splitlines()]
    second_events = [
        r["event"] for r in records
        if r["request_id"] == "R-X-REPLAY-42"
    ]
    assert second_events == [
        "request_received", "validation_passed",
        "ticket_intent_replayed", "response_completed",
    ]
    assert "打印机离线" not in logs.getvalue()

print("同一工单；一次创建；一次外部效果；重放与读回符合约定。")
```

BODY 是 X01 展示的四字段输入，headers() 加入固定创建 key 和传入的 request ID。这里没有隐藏一个网络服务：client 是 TestClient；它调用本地 API 应用，数据库使用真实临时 SQLite，partner 的 HTTPX request 由 MockTransport 交给有状态 Fake。

直接运行这段程序即可得到断言结果。若把重放 payload 中的 subject 改掉，预期应改为 409，后面的“同一成功结果”断言不再适用。这一次修改已经足够暴露是否理解 key 与输入的绑定，不必反复添加同类题。

SQL 查询为什么还要关心物理字段名？

API 使用 ticket_status，数据库使用 status。需要直接检查事实时，查询必须符合真正的 schema：

SQL

```
SELECT ticket_id, priority, status AS ticket_status, assignment_status
FROM tickets
WHERE ticket_id = :ticket_id;
```

把 T-42 作为参数绑定到 `:ticket_id`，得到本地行。`AS ticket_status` 是结果列别名，没有给数据库增加同名物理列。SQL 查询与 API 的公开映射在这里接上了，但并未变成同一层。

查询的连接目标同样重要。旧实验数据库里查不到 T-42，不能证明当前 API 没保存；在正确文件里查到 T-42，也不能证明调用者取得了正确 response。因此排障报告要说明连接的文件与本次环境，而不是只贴一张“有一行”的结果。

为什么还要保留小测试?

上面的组合实验已经走过多个组件，但不适合解释每一种失败。例如转换输入错了，最先失败可能是最后的 201 断言；你仍需要回到 parse_priority 的输入才能知道它为何没有发请求。

纯函数测试可固定 `"3"`、`"urgent"` 与 `"-3"`，快速判断转换和范围；事务测试在 event INSERT 处制造拒绝，检查票与计数器一起撤回；partner 测试模拟“提交后丢回复”，直接核 calls=2、effects=1；AI 测试固定坏引用和越权建议，判断工具次数是否为零。每一个小实验都保留要检查的实现，只替换与当前问题无关的不稳定边界。

测试用例的名称或 case_id 表示“哪一种条件”；request_id 表示“哪一次 HTTP 处理”。同一个测试可以运行很多次，也可以发出四个不同请求。把它们写成相同字符串不会让独立活动变成一次真实运行。

运行配套测试时：

```powershell
POWERSHELL
& $xPython -m pytest -q
if ($LASTEXITCODE -ne 0) { throw '至少一项已登记行为不符合预期。' }
```

结果只对这套实现与这些输入成立。TestClient 的通过不证明端口、DNS 或容器网络；模拟外部结果也不证明真实供应商的 TLS、凭据、容量和服务水平。它们的价值在于让本地行为可以反复检查，而不是扩大结论。

从进程内协作扩大到两个真实进程

`host_integration_check.py` 另启动 API 与模拟合作方两个 Uvicorn 进程，分别监听动态 loopback 端口。HTTPX 真正经过本机连接发送 JSON，API 在系统临时目录中使用自己的 tickets.db，合作方使用另一文件 receipts.db。

```powershell
POWERSHELL
& $xPython .\host_integration_check.py
if ($LASTEXITCODE -ne 0) { throw '双进程主机集成没有通过。' }
```

它验证普通创建、政策请求、API 重启后读回同一文件；随后验证合作方重启后的 receipt 仍在，以及应用备份恢复后的指定业务路线。程序结束时关闭自己的进程并清理自己的临时目录，不终止未知进程。

这比 TestClient 增加了本机 socket、端口、进程启动与重启证据，仍没有经过 Compose service DNS 或 named volume。观察范围扩大到哪里，结论就写到哪里。

一次 UAT 为什么有四个请求编号？

UAT 是 User Acceptance Testing，这里把它做成一条合成客户路线：创建、同意图重放、GET 读回、请求政策草稿。它不是打开服务首页，也不是代替真实客户验收。

`uat.py` 接受已运行 API 的 loopback 地址。它默认 `--initial-create-mode fresh`：第一条请求必须是新创建；接着第二条才是 replay。已有 T-42 上切换 AI 场景或恢复配置时，用 `--initial-create-mode replay`，明确第一条也应取回旧意图。模式选错就失败，不自动把错误基线接受为成功。

创建、重放、查询和政策请求分别使用 request ID。UAT 检查创建结果、重放结果和 GET 的全部公开工单字段相等，再检查政策状态、原因、来源与 tool_called。调用次数与外部效果还需由对应 Fake 的记录或专门测试补足；仅运行 uat.py 不会凭空知道合作方内部状态。

例如在普通已完成的 T-42 上，将 Fake Model 配置改成 close_proposal 后，UAT 的期望是：既有意图 replay、工单仍 open、政策 needs_human、原因 tool_permission_denied、tool_called=false。恢复 grounded 配置后再发新请求，应重新取得 accepted_draft。这里“恢复配置”不等于撤销工单，也不重新创建同一意图。

日志怎样解释一次处理，又怎样保护输入？

正常 R-X-42 应按顺序记录 request_received、validation_passed、ticket_transaction_committed、partner_attempts_completed、assignment_transaction_committed、response_completed。重放请求走的是另一条较短事件链，政策请求又有自己的 policy_candidate_checked。

日志保存 request ID、合成 ticket ID、阶段、结果、状态码、原因、允许的版本与计数。它不保存 subject、完整请求、prompt、模型草稿、token 或原始异常。排障不需要把每个输入长期复制进日志；需要的是足以连接当前处理的允许字段。

日志也会有缺陷。若出现“已 commit”事件，数据库却没有目标行，下一步核对事件时机与查询目标，不能让日志文字覆盖持久事实。若只收集到了最后 response_completed，也不能反推所有中间动作一定发生。

同一 request ID 帮我们把线索接起来，数据行和副作用计数帮我们判断线索是否支持真实结果。这样，测试、日志、查询与 UAT 才是合作的证据，不是一串可以互相替代的绿色标签。

X04 | 换掉运行实例以后，怎样确认还能服务和恢复？

先追 identity、volume、readiness、create/read 和 restore；能响应、已 ready、业务正确是三件不同的事。

前面已经看见 Python 程序、HTTP、数据库、外部模拟服务和模型分支怎样协作。现在把这些程序交给另一位操作者运行。新的问题不是业务规则换了，而是：他启动的是哪份程序，连接哪份数据，失败以后能否按说明回到可用状态？

本章先固定一组可观察输入和输出：输入是旧 / 新 image、启动配置、应用与合作方两个 volume，以及 B1 备份；输出是新实例接回同一张 T-42、`/live` 与 `/ready` 的分层结果、业务 UAT、隔离恢复 readback 和回滚证据。后面的每个命令都只回答其中一项，不能用一个绿色状态替代整组结果。

先从“删除旧 container，再创建一个新 container”看起。若工单只写在旧实例可写层中，旧实例消失后新实例找不到它。要让数据生命周期独立于实例，必须把文件放在明确的持久存储位置，并让新实例重新取得它。

从源码到文件路径，中间有哪些真实对象？

Dockerfile、源码和依赖记录是 build 输入。build 产生 image；Compose 在启动时另外指定环境变量、端口和挂载；container 是从 image 创建的一次运行实例。

读下面的图时只追三条线：build 输入怎样变成 image，image 加配置怎样变成 container，container 的 mount 怎样指向真正打开的数据库文件。再看两个数据库分别由谁拥有，便不会把运行实例、挂载位置和业务事实混成一个对象。

构建产物

源码 + Dockerfile + requirements → build → image

API 运行分支：应用数据

image + 启动配置
↓
API container
↓
/data 挂载应用 volume ticket-data
↓
API 打开 /data/tickets.db

合作方运行分支：合作方数据

image + 另一份启动配置
↓
fake-partner container
↓
/partner-data 挂载合作方 volume partner-data
↓
模拟合作方打开 /partner-data/receipts.db

API 数据与模拟合作方数据放在两个不同的 volume。二者即使在同一台电脑上，也由不同程序写入。应用的 backup 工具只处理 tickets.db；它没有因此取得合作方事实的所有权。

named volume 是 Docker 管理的持久存储对象，`/data` 是 container 内看到的挂载位置，`tickets.db` 才是应用打开的文件。它们不是同一个名称的三种写法，也不能把 `/data` 当成 Windows 上的某个固定目录。

Compose 的相关切片如下；它用来说明关系，实际运行使用配套完整 compose.yaml，包括权限、healthcheck 和其他字段：

先找四个字段：`X_DATABASE_PATH` 是 API 实际打开的文件，`X_PARTNER_BASE_URL` 是容器间服务名，`ports` 是主机到容器的入口，两个 `volumes` 是两份持久状态的挂载。下面这段 YAML 的输出是连接关系；它本身不能证明数据库里已经存在 T-42 或 PA-1。

YAML

```yaml
services:
  api:
    environment:
      X_DATABASE_PATH: /data/tickets.db
      X_PARTNER_BASE_URL: http://fake-partner:9000
    ports:
      - "127.0.0.1:18080:8000"
    volumes:
      - ticket-data:/data
  fake-partner:
    environment:
      X_FAKE_PARTNER_DATABASE_PATH: /partner-data/receipts.db
    volumes:
      - partner-data:/partner-data
volumes:
  ticket-data:
  partner-data:
```

Windows 客户端连接 host port 18080，再转到 API container 的 8000。API 访问 fake-partner:9000，使用 Compose 的服务名找到另一个 container；API container 内的 localhost 只指它自己。

同一个 image 可以取得不同配置。端口和 volume 声明不会因 build 完成就自动进入 image；配置也不会使已有数据变成它期望的内容。重新创建实例后，要核对实际 mount，再 GET 同一工单，才能说明新实例确实接上了原数据。

哪份程序配哪份数据，不能只凭版本名猜

给已知可用 image 起一个便于讨论的标签 D1，给待验收 image 起标签 D2。实际记录必须保存 Docker 返回的完整 local image ID，并检查运行中 container 的 `.Image` 是否相同。可移动的 tag 不能独立承担这个证据；只有真实使用 registry 且取得 RepoDigest 时，才登记仓库 digest。

配置也有身份。C1 是基础 Compose 展开后的指纹，C2 是按指定顺序合并 overlay 后的指纹。配套脚本对 `docker compose config --format json --output ...` 产生的文件字节求 hash，避免 PowerShell 管道编码改变字节。当前项目不包含秘密；若扩展含秘密的配置，不应公开输出或把含秘密的完整模型当共享资料。

程序还声明 expected schema version，SQLite metadata 保存 actual schema version。两者都为 1，表示这项版本条件相容。本项目 readiness 同时尝试访问 tickets；它并没有逐项验证全部列、约束、索引或所有写入能力。因此“版本匹配”仍要与业务测试配合。

当数据已升级到旧程序不能使用的 schema 时，把 image 切回 D1 不会自动把数据变回去。回滚前必须判断数据影响：旧程序仍能读写这份数据吗，需要独立的数据恢复吗？这个问题比“旧 tag 还在不在”更接近实际恢复条件。

能响应、准备接单、业务正确是不同的观察

container running 表示主进程仍在运行。`GET /live` 检查应用能否回答最小请求；`GET /ready` 检查本实例的必要数据读取和 schema 版本条件。二者都不能代替创建工单与读回。

按证据层次读这一段：进程存在是第一层，`/live` 是最小响应，`/ready` 是数据读取与 schema 条件，UAT 才检查创建、重放、GET 与政策结果，backup 又是另一条恢复证据。每层的输入和输出不同，不能把 `/ready` 的 200 当作整套业务已经通过。

本项目的模拟合作方是可降级依赖，因此它不可用不直接令核心 readiness 失败。API 仍可以保存工单并转人工。若真实客户合同规定“没有合作方确认就不能接单”，readiness 与业务策略需要按那个合同重新设计，不能照搬本例。

> container running
> → 还要检查 /live
> → 还要检查 /ready
> → 还要运行创建、重放、查询与政策的 UAT
>
> UAT 通过
> → 仍需要另外验证 backup 能否恢复

启动后的健康状态需要等待才能稳定，不能立刻读一次失败就断言程序永远不可用。配套 `healthcheck.py --wait-seconds 15` 会在有限预算内轮询并在失败时退出。等待动作有上限，且只表示检查了相应 endpoint；它不吞掉失败，也不把时间耗尽写成通过。

普通、降级和版本回归为什么需要不同基线？

第一次普通 UAT 要求应用库为空，模拟合作方的 receipt store 也为空。这样 T-42、PA-1、一次创建和一次派单的含义才明确。先完成普通创建，再在这张票上验证 replay 与只读 AI 场景，可以共享这份已说明的状态。

合作方降级实验则使用另一 Compose project、另一端口和两份新的 volume，创建 key 为 ticket-create:Printer-7:uat-degraded-001；overlay 让合作方每次返回 500。它不是普通创建的 retry。若拿普通已收束的 key 去试，API 会直接返回旧结果，根本没有访问故障合作方，便无法证明降级。

配套 RUNBOOK 把普通应用 volume 记作 V1，独立降级的应用 volume 记作 V2。D2 回归场景在明确结束旧合成基线后重新创建应用 volume，记作 V3；即使 Docker name 字符串与 V1 相同，也是另一个对象生命周期。合作方 volume 也各有自己的生命周期，需要一同登记，但不与应用 volume 混称。

反复调用同一意图用原 key；创建独立场景用独立 key；重新开始空基线则要确认两边状态都与登记一致。这三件事不同，不能靠修改 request ID 解决基线问题。

一份 B1，怎样从文件变成可恢复证据？

普通路线完成后，应用数据库含 T-42/open/partner_confirmed 和对应事件。备份工具利用 SQLite 的 backup API，把数据库的一致状态写入独立文件，再记录 hash、schema version、程序身份与配置指纹。

这条 trace 的输入是 B1 manifest 与备份源文件，输出只到隔离恢复库的完整性、schema 和 T-42 读回；它不重建合作方数据库，也不替代 image 回滚。带着这个边界再读下面的步骤，就能知道每一步新增了哪份证据。

一致性表示备份取得一组相互协调的数据库事实。随手复制正在变化的数据库文件，不自动拥有这个保证；SQLite 提供的在线备份接口专门处理源数据库与目标数据库之间的数据复制。

看到 B1 目录、文件大小和 hash，只能确认备份输入存在且未被意外改动。恢复还要继续：

> 核对 B1 manifest 与备份源文件 hash
> → restore 到一个原本不存在的新数据库目标
> → 打开恢复文件，PRAGMA integrity_check 为 ok
> → 核对 metadata 的 schema version
> → 用隔离 API 打开它
> → GET T-42 并核对设备、主题、priority=3、ticket_status=open

这条路线叫 restore readback。它没有新建工单，没有重新验证外部降级，也没有回滚 image。恢复文件的物理布局可以与备份文件不同，所以配套校验的是备份源 hash、恢复后的数据库完整性与登记字段，不把“恢复文件字节 hash 必须相等”作为数据库恢复的定义。

同样，schema version=1 不等于全部 DDL 已检查；读回四个关键字段不等于所有业务规则都成立。报告要写明这次观察了什么，后续才知道还需要补哪一项。

配套 `Invoke-IsolatedRestoreDrill.ps1` 检查来源 image/config，建立带精确 label 的新 volume，用隔离 API 完成这条 readback。默认不联系合作方；失败时保留明确标记的资源供检查，成功时只清理它自己创建且 label 匹配的对象。它拒绝覆盖已有数据库目标，避免把练习恢复写到原数据上。

恢复应用，为什么不能让外部世界一起回到过去？

假设备份中的 T-42 对应合作方 PA-1。API 停机后，从 B1 恢复 tickets.db；这不会撤销合作方已经创建的 PA-1。如果合作方继续收到 assignment:T-42，它应查询自己的 receipt，交回原结果。

这里有两个状态所有者：B1 只含应用的 `tickets.db`，合作方的 `receipts.db` 独立保存 PA-1。恢复前者只能让本地重新认识旧意图，不能让外部世界回到过去；后面的 replay 正是用这个边界核对原 key 是否仍指向同一效果。

新版模拟合作方因此把 key、payload 摘要、派单编号和计数持久保存在自己的 receipts.db。它的进程重启后仍能识别旧意图：相同 key 与内容返回 PA-1，effect 不增加；新意图得到 PA-2。这不是给真实供应商作保证，而是把本实验依赖的幂等前提真正保存下来。

若只用内存计数器，重启后又从 PA-1 开始，新 T-43 也可能收到 PA-1；应用中已有该结果编号，就会暴露冲突。换随机编号只能躲开这一次编号相撞，仍不能恢复“旧 key 是否已经执行”的知识。缺失的是持久 receipt，不是编号看起来不够独特。

因此应用 B1 不包括模拟合作方数据库。正常恢复应用时保留合作方原 store，不把它重置为空，也不跟着 B1 回滚。若合作方自己的持久状态丢失，这条已登记恢复路线的前提不再成立；应先核对外部事实与其恢复机制，不能宣称恢复本地备份已经修好了对方。

恢复后写入，要另做一次什么验证？

readback 通过以后，可以在该隔离恢复环境继续验证明确列出的客户路线。本例 B1 只有 T-42，所以下一个新意图应得到 T-43：

起点先写清：应用库有 T-42，合作方已有 PA-1，下一编号是 43。旧 key 的输出应是同一 T-42 且不增加效果；新 key 才产生 T-43 与 PA-2。这样编号、意图和外部效果不会被“恢复成功”四个字压成一件事。

各自拥有数据的恢复基线

恢复出的应用库：T-42 → PA-1，next_ticket_number=43

合作方原 store：assignment:T-42 → PA-1

已有创建意图

重放旧创建意图
↓
同一 T-42，200，不新增派单

新的创建意图

新 key 创建新工单
↓
T-43
↓
合作方保存新的 assignment:T-43 → PA-2
↓
本地第二笔事务保存确认
↓
201，再显式重放为200
↓
GET 读回全部公开字段
↓
政策请求返回有来源的草稿

恢复后的新写入之所以能用 PA-2，不是应用备份保存了合作方计数，而是合作方独立状态一直存在。两份数据库之间没有共同回滚，测试只是核对它们按接口约定继续协调。

主机集成实际使用两个本机进程来检查这组关系。容器手册则提供 `-RunRestoredUat`：把新的隔离恢复 API 接入已核对的合成合作方网络，先验旧意图 replay，再验新 T-43 的创建、重放、读回和政策。它会在模拟合作方产生一项新效果；清理隔离 API 并不撤销该效果。

因此，这里的“隔离”只覆盖恢复出的应用库，合作方仍是共享的。演练后，原应用库的下一编号仍可能是 T-43，但合作方已经保存了 `assignment:T-43 → PA-2`；原库不能再把这个编号作为独立新意图继续创建。要继续或重复实验，应按 RUNBOOK 重新规划双方基线，不能只清理隔离 API 就把合作方当作从未执行。

readback 与恢复后 UAT 分别报告。当前这组恢复后 UAT 是已列出的子集，不包含合作方 always_500、close_proposal 权限拒绝或版本回滚。若交付要求这些路线在恢复后也成立，就给它们明确基线和期望，再分别运行，不由一次普通成功外推。

D2 可以 ready，为什么仍然不能采用？

配套提供一个受控坏版本：build variant=uat-regression。它仍能读取 schema，因此 /live 与 /ready 可以返回 200；但合法 POST /tickets 会在意图事务之前返回 503，detail=training_create_regression。

这条回归的输入是 D2/C2 与明确结束旧基线后的 V3；期望输出是 `training_create_regression`、没有数据变化，切回登记的 D1/C1 后再完成业务验收。以下并发 pending 的说明是状态保护的补充，不是 D2 回滚步骤。

这个版本用来说明健康检查没有枚举全部业务规则。观察到这个具体 detail，同时确认无新工单、事件和派单，编号也未消耗，才说明命中了本次受控回归。只看到任意 503，也可能是另一种数据库故障。

> D2/C2，schema仍为1
> → ready通过
> → 创建UAT命中training_create_regression
> → D2不采用，保存身份与失败证据
> → 确认没有破坏性数据变化，D1仍兼容当前schema
> → 切回D1/C1并核对container实际Image
> → readiness重测
> → 新创建、同意图重放、GET与政策重新验收

这是代码与配置的回滚，不需要顺手把 B1 覆盖到当前数据上。实验的 D2 没有执行破坏性 migration；因此它不能证明任何 schema 迁移都可用同样方式撤销。

配套在 V3 上完成 D2/D1 这条路线，未声称找回被明确清理的 V1。切回 D1 后仍用此前失败意图的 key 和 payload，下一张仍应是该新数据库的 T-42，证明 D2 的事务前拒绝没有偷偷消耗编号。

若两份并发处理刚好对同一个 pending 作不同决定，最终 response 还要以已经提交的数据库状态为准。先完成的合法 finalization 胜出，后完成者记录 assignment_finalize_replayed，不再虚构第二次 commit。回滚和恢复不能跳过应用原有的状态保护。

让另一位操作者可以按说明做完

一份 runbook 要把动作连到可观察结果。它需要说明在哪个实验目录、哪个 project 和端口上运行，用哪些 image/config/data，成功应出现什么，失败在何处停止，以及如何保留或清理精确对象。

配套 RUNBOOK 的完整路线依次建立普通基线，验证 AI 场景与独立降级，检查 schema mismatch，生成 B1，完成隔离 readback，再按需要运行恢复后 UAT 和受控 D2/D1 回滚。它不使用模糊的 prune 命令清理其他项目。

运行前确认 Docker Client 能联系 Server、Compose 可解析；遇到需要安装组件或重启系统时，应先处理自己的环境准备，再回到手册。阅读本章不要求同时完成这些操作。具体命令输出、Docker Desktop 和 Compose 支持随版本变化，实际操作时使用当日官方手册核对。

尤其要知道，`docker compose down` 默认保留 named volume；`down --volumes` 会丢弃此 project 的相关 volume，包括这里的应用与模拟合作方数据。它只能用于已经检查 label、确认内容与恢复安排、明确决定结束的合成实验。另建同名 volume 得到的是空的新对象，不会因为名称相同而找回数据。

每份最终记录写明目标环境、对象身份、输入、实际输出与结论范围。没有运行的路线保持未验证；失败路线按失败登记；通过 readback、通过恢复后指定 UAT、通过 rollback 分别写清。别人的环境通过，也不会自动改变当前目标环境的证据。

关系收束 | 从一个现象找到下一项有用证据

经过四章，系统没有变成一条“A 服务→B 服务→C 服务→D 服务→E 服务→F 服务”的流水线。Python 在多个地方组织控制，HTTP 连接进程，数据库保存事实，外部依赖按业务需要调用，AI 是独立的候选生成分支，运行环境则承载它们。

同一张 T-42 给出三条可以长期找回的关系：

请求关系：

文字 → 本地值 → HTTP → API → 公开结果

状态关系：

核心事务 → 本地pending → 外部结果 → 第二事务 → 可读回状态

恢复关系：

程序身份 + 配置 + 持久对象 → 实例运行 → 业务验证 → 隔离恢复与再验证

这些关系在一个具体问题上相遇。例如看到 503，先用请求编号找本次处理，再查是否已经有本地 pending 和外部 receipt；确认交接位置后选择恢复动作，最后用同一意图的调用次数、效果和读回结果证明动作没有扩大错误。

当前最早不清楚的关系	回到哪里继续
文字怎么变成值，函数下一步为何这样走	A 册输入、分支、异常与函数；本书 X01 的客户端到 API

当前最早不清楚的关系	回到哪里继续
没有请求、没有 response，或 API 拒绝了输入	B 册请求/响应与 HTTPX；本书 X02 的连接与校验边界
行、事件和计数为何必须一起变化	C 册关系、事务与日志；本书 X01 的两笔事务
超时以后能否再发，外部是否已经执行	D 册预算、稳定意图与人工出口；本书 X02 的 pending 恢复
草稿有引用却不能执行动作	E 册依据、用途与授权；本书 X01 的独立政策请求
程序重建后接上哪份数据，备份能证明什么	F 册运行对象与恢复；本书 X04 的两个持久状态所有者
绿色测试到底保护了哪项客户承诺	本书 X03 的 response、行、效果与日志组合

重要的不是把表里所有名称背下来，而是在面对一个实际输入、状态或失败时，能够指出它所属的对象、下一次交接和最有用的证据。遗忘命令细节可以查；若无法解释“这次恢复改变哪份状态、为什么可以这样做”，就应回到那一段关系重新追踪。

查用来源

以下为持续更新的官方操作资料，复核日期为 2026-09-07；具体版本敏感写法在真正运行时再次核对。本书配套的字段、状态和固定场景由随书源码与测试定义，不由文档更新自动改变。

- FastAPI Testing：TestClient 与接口测试入口。
- HTTPX Timeouts：连接、读取、写入与连接池等待的区别。
- SQLite Online Backup API：一致数据库备份与恢复输入。
- Docker Volumes：volume 与 container 生命周期，以及备份恢复操作。

固定例子由本书与配套完整提供。真实客户的数据、真实供应商的约定、真实模型的效果和另一台机器的运行状态，仍需各自的证据与授权范围来回答。

本册总览（需要时再回看）

这里保留整册地图、章节关系和查用入口；第一次阅读不需要先读完。

本册知识结构总览

先分清四个相互配合的问题。它们是观察同一系统的方式，不是请求依次经过的四项服务。

遇到的问题	本册沿什么关系回答
X01：这张工单究竟怎样产生？	从文字追到请求、事务、外部结果和读回；分清谁产生结果、谁保存事实
X02：为什么都显示失败，下一步却不同？	比较本应发生与实际发生的交接，判断失败前已经留下什么
X03：测试是绿的，为什么还不能说整套应用都对？	让 response、数据行、调用计数和日志共同支持一项具体承诺
X04：程序能启动以后，怎样证明可以交接和恢复？	把程序身份、配置、持久数据、业务验收与恢复接在一起

支持应用本身的关系如下。图中数据库在应用旁边，运行环境承载所有程序；它们都不是最后再处理一次请求的“下一站”。

进入应用的请求

客户端
↓ 创建 / 查询请求
工单 API

应用持久数据关系

工单 API
↓ 事务与查询
应用 SQLite
tickets.db

返回客户端的方向

工单 API
↓ 公开结果
客户端

合作方调用及独立的数据归属

工单 API
↓ 派单请求
模拟合作方
自己的 receipt 数据

另一条政策请求路线

工单 API
↓
另一次政策请求
↓
读取工单与获准资料
↓
Fake Model 提出草稿
↓
应用检查后交人核对

运行承载关系，不是下一站

运行环境为 API 和模拟合作方提供进程、配置、端口与持久目录。

备份与恢复的操作边界

备份与恢复操作处理已经明确归属的数据，不替任何业务层作决定。

这张图有两种不同的“保存”。应用保存工单和处理事件；合作方保存自己的派单结果。二者经过请求协调，但不是一个数据库事务。模型分支又不同：它产生的草稿只交回调用者，本例没有把它写成工单的新事实。

这里的 T-42 与前几册是什么关系？

固定设备叫 `Printer-7`，主题为“打印机离线”，外部优先级文字是 `"3"`。本项目只使用合成数据，并要求请求显式标记 `data_classification="synthetic"`。标记是输入合同的一部分，不是程序能自动判定任意文字是否来自真实客户的证明。

X 使用自己的独立数据库。只有从空数据库开始、编号计数为 42 时，第一张成功创建的工单才是 `T-42`。后续新意图会得到 T-43 等编号。B、C 即使也用过 T-42，也没有与这里共享数据；D、E 的 `LOCAL-0042` 则属于另一套本地案件命名，不能直接替换本项目的工单号。

本配套的客户端是 Python 程序，没有附带网页界面。读者可以把公开结果交给网页显示，但本书的可运行验证只声称检查实际存在的程序、接口和数据。